Assignment

Bachelor of Computer Applications

Semester III

Database Management System

Topic: DATABASE CREATION AND TABLE IMPLEMENTATION

BY

Name: M Blessy

Registration Number: 2511021240067

DEPARTMENT OF COMPUTER APPLICATIONS

Alliance university

Chikkahagade cross, Anekal, Bangaluru 562106

DATABASE CREATION AND TABLE IMPLEMENTATION

1. Aim

To create a Hospital Management System database and implement the required tables using SQL Data Definition Language (DDL) commands, ensuring that the tables and their relationships are properly defined.

2. Software Used

- Database Management System: MySQL
- Interface: MySQL Workbench
- Query Language: SQL

3. Database Description

The Hospital Management System manages hospital information, including departments, doctors, patients, appointments, rooms, admissions, prescriptions, medicines, laboratory tests, and bills.

The database contains the following 11 tables:

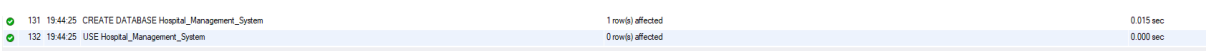
1. Department
2. Doctor
3. Patient
4. Appointment
5. Room
6. Admission
7. Prescription
8. Medicine
9. Prescription_Detail
10. Lab_Test
11. Bill

4. SQL DDL Commands

4.1 Create the Database

```
CREATE DATABASE Hospital_Management_System;
USE Hospital_Management_System;
```

Output: The database is created and selected for use.



Note: The original setup script also contains DROP DATABASE IF EXISTS. Do not rerun that statement simply to capture this screenshot, because it deletes the existing database.

4.2 Create the Department Table

```
CREATE TABLE Department (
   department_id INT PRIMARY KEY,
   department_name VARCHAR(50) NOT NULL,
   floor_no INT,
   phone_no VARCHAR(15),
   head_doctor_id INT
);
```

The table stores department names, floor numbers, contact numbers, and the department head's doctor ID.

4.3 Create the Doctor Table

```
CREATE TABLE Doctor (
   doctor_id INT PRIMARY KEY,
   name VARCHAR(60) NOT NULL,
   specialization VARCHAR(50),
   department_id INT,
   phone_no VARCHAR(15),
   email VARCHAR(60),
   experience_years INT,
   consultation_fee DECIMAL(8,2),

   CONSTRAINT fk_doctor_department
      FOREIGN KEY (department_id)
      REFERENCES Department(department_id)
);
```

The table stores doctor information. The foreign key connects each doctor to a department.

4.4 Add the Department Head Relationship

```
ALTER TABLE Department
ADD CONSTRAINT fk_department_head_doctor
FOREIGN KEY (head_doctor_id)
REFERENCES Doctor(doctor_id);
```

This establishes a relationship between a department and its head doctor. It is added after creating Doctor because the two tables reference each other.

4.5 Create the Patient Table

```
CREATE TABLE Patient (
   patient_id INT PRIMARY KEY,
   name VARCHAR(60) NOT NULL,
   dob DATE,
   gender VARCHAR(10),
   phone_no VARCHAR(15),
   address VARCHAR(150),
   blood_group VARCHAR(5),
```

```
    emergency_contact VARCHAR(15)
);
```

The table stores patient details, including date of birth, contact information, address, and blood group.

4.6 Create the Appointment Table

```
CREATE TABLE Appointment (
    appointment_id INT PRIMARY KEY,
    patient_id INT,
    doctor_id INT,
    appointment_date DATE,
    appointment_time TIME,
    status VARCHAR(15) DEFAULT 'Scheduled',
    reason VARCHAR(150),

    CONSTRAINT fk_appointment_patient
        FOREIGN KEY (patient_id)
        REFERENCES Patient(patient_id),

    CONSTRAINT fk_appointment_doctor
        FOREIGN KEY (doctor_id)
        REFERENCES Doctor(doctor_id)
);
```

The table records appointments between patients and doctors.

4.7 Create the Room Table

```
CREATE TABLE Room (
    room_id INT PRIMARY KEY,
    room_number VARCHAR(10) NOT NULL,
    room_type VARCHAR(20),
    floor_no INT,
    daily_charge DECIMAL(8,2),
    status VARCHAR(15) DEFAULT 'Available',

    CONSTRAINT uq_room_number UNIQUE (room_number)
);
```

The table stores room details, daily charges, and room availability. The UNIQUE constraint prevents duplicate room numbers.

4.8 Create the Admission Table

```
CREATE TABLE Admission (
    admission_id INT PRIMARY KEY,
    patient_id INT,
    room_id INT,
    doctor_id INT,
    admission_date DATE,
    discharge_date DATE,
    admission_reason VARCHAR(150),
    status VARCHAR(15) DEFAULT 'Admitted',
```

```
    CONSTRAINT fk_admission_patient
        FOREIGN KEY (patient_id)
        REFERENCES Patient(patient_id),

    CONSTRAINT fk_admission_room
        FOREIGN KEY (room_id)
        REFERENCES Room(room_id),

    CONSTRAINT fk_admission_doctor
        FOREIGN KEY (doctor_id)
        REFERENCES Doctor(doctor_id)
);
```

The table records patient admissions, assigned rooms, responsible doctors, and discharge details.

4.9 Create the Prescription Table

```
CREATE TABLE Prescription (
    prescription_id INT PRIMARY KEY,
    appointment_id INT,
    patient_id INT,
    doctor_id INT,
    date_issued DATE,
    notes VARCHAR(200),

    CONSTRAINT fk_prescription_appointment
        FOREIGN KEY (appointment_id)
        REFERENCES Appointment(appointment_id),

    CONSTRAINT fk_prescription_patient
        FOREIGN KEY (patient_id)
        REFERENCES Patient(patient_id),

    CONSTRAINT fk_prescription_doctor
        FOREIGN KEY (doctor_id)
        REFERENCES Doctor(doctor_id)
);
```

The table stores prescription information and links prescriptions to appointments, patients, and doctors.

4.10 Create the Medicine Table

```
CREATE TABLE Medicine (
    medicine_id INT PRIMARY KEY,
    medicine_name VARCHAR(60) NOT NULL,
    manufacturer VARCHAR(60),
    unit_price DECIMAL(8,2),
    stock_quantity INT
);
```

The table stores medicine names, manufacturers, prices, and available stock.

4.11 Create the Prescription_Detail Table

```
CREATE TABLE Prescription_Detail (
   prescription_detail_id INT PRIMARY KEY,
   prescription_id INT,
   medicine_id INT,
   dosage VARCHAR(30),
   duration_days INT,
   quantity INT,

   CONSTRAINT fk_detail_prescription
      FOREIGN KEY (prescription_id)
      REFERENCES Prescription(prescription_id),

   CONSTRAINT fk_detail_medicine
      FOREIGN KEY (medicine_id)
      REFERENCES Medicine(medicine_id)
);
```

This table connects prescriptions with medicines and records dosage, duration, and quantity.

4.12 Create the Lab_Test Table

```
CREATE TABLE Lab_Test (
   test_id INT PRIMARY KEY,
   patient_id INT,
   doctor_id INT,
   test_name VARCHAR(60),
   test_date DATE,
   result VARCHAR(200),
   status VARCHAR(15) DEFAULT 'Pending',

   CONSTRAINT fk_lab_patient
      FOREIGN KEY (patient_id)
      REFERENCES Patient(patient_id),

   CONSTRAINT fk_lab_doctor
      FOREIGN KEY (doctor_id)
      REFERENCES Doctor(doctor_id)
);
```

The table records laboratory tests, including the patient, ordering doctor, test date, result, and status.

4.13 Create the Bill Table

```
CREATE TABLE Bill (
   bill_id INT PRIMARY KEY,
   patient_id INT,
   appointment_id INT NULL,
   admission_id INT NULL,
   bill_date DATE,
   total_amount DECIMAL(10,2),
   payment_status VARCHAR(15) DEFAULT 'Pending',
   payment_mode VARCHAR(20),

   CONSTRAINT fk_bill_patient
      FOREIGN KEY (patient_id)
      REFERENCES Patient(patient_id),
```

```
    CONSTRAINT fk_bill_appointment
        FOREIGN KEY (appointment_id)
        REFERENCES Appointment(appointment_id),

    CONSTRAINT fk_bill_admission
        FOREIGN KEY (admission_id)
        REFERENCES Admission(admission_id)
);
```

The table stores billing and payment information. The appointment and admission IDs are nullable because a bill may not be associated with both types of records.

5. Table Relationships

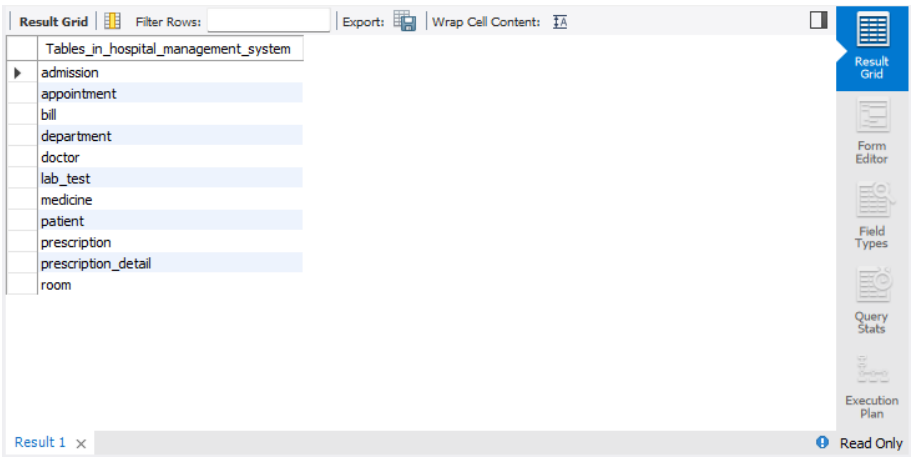
The database uses primary keys and foreign keys to connect related tables.

- Department and Doctor: Each doctor can belong to a department, and a department can have multiple doctors.
- Department and Doctor (department head): A department can reference a doctor as its head.
- Patient and Appointment: A patient can have multiple appointments.
- Doctor and Appointment: A doctor can attend multiple appointments.
- Patient, Room, and Doctor with Admission: Admissions identify the patient, assigned room, and responsible doctor.
- Appointment, Patient, and Doctor with Prescription: Prescriptions refer to the related appointment, patient, and doctor.
- Prescription and Medicine: Prescription_Detail connects prescriptions with medicines.
- Patient and Doctor with Lab_Test: Each lab test references a patient and doctor.
- Patient, Appointment, and Admission with Bill: Bills reference patients and may reference appointments or admissions.

6. Verify the Database Tables

Run the following query to display all tables in the database.

```
USE Hospital_Management_System;
SHOW TABLES;
```



7. Verify the Table Constraints

Run the following query to inspect the constraints defined in the database.

```
SELECT
   TABLE_NAME,
   CONSTRAINT_NAME,
   CONSTRAINT_TYPE
FROM information_schema.TABLE_CONSTRAINTS
WHERE CONSTRAINT_SCHEMA = 'Hospital_Management_System'
ORDER BY TABLE_NAME, CONSTRAINT_TYPE;
```

TABLE_NAME	CONSTRAINT_NAME	CONSTRAINT_TYPE
admission	chk_admission_status	CHECK
admission	fk_admission_doctor	FOREIGN KEY
admission	fk_admission_patient	FOREIGN KEY
admission	fk_admission_room	FOREIGN KEY
admission	PRIMARY	PRIMARY KEY
appointment	chk_appointment_status	CHECK
appointment	fk_appointment_doctor	FOREIGN KEY
appointment	fk_appointment_patient	FOREIGN KEY
appointment	PRIMARY	PRIMARY KEY
bill	chk_bill_amount	CHECK
bill	chk_payment_status	CHECK
bill	fk_bill_admission	FOREIGN KEY
bill	fk_bill_appointment	FOREIGN KEY
bill	fk_bill_patient	FOREIGN KEY
bill	PRIMARY	PRIMARY KEY
department	fk_department_head_d...	FOREIGN KEY
department	PRIMARY	PRIMARY KEY
department	uq_department_name	UNIQUE
doctor	chk_doctor_fee	CHECK
doctor	chk_doctor_experience	CHECK
doctor	fk_doctor_department	FOREIGN KEY
doctor	PRIMARY	PRIMARY KEY
doctor	uq_doctor_email	UNIQUE
lab_test	chk_lab_status	CHECK
lab_test	fk_lab_doctor	FOREIGN KEY
lab_test	fk_lab_patient	FOREIGN KEY
lab_test	PRIMARY	PRIMARY KEY
medicine	chk_medicine_price	CHECK
medicine	chk_medicine_stock	CHECK
medicine	PRIMARY	PRIMARY KEY
patient	PRIMARY	PRIMARY KEY
prescription	fk_prescription_appoint...	FOREIGN KEY
prescription	fk_prescription_doctor	FOREIGN KEY
prescription	fk_prescription_patient	FOREIGN KEY
prescription	PRIMARY	PRIMARY KEY
prescription...	chk_prescription_duration	CHECK
prescription...	chk_prescription_quantity	CHECK
prescription...	fk_detail_medicine	FOREIGN KEY

prescription...	fk_detail_prescription	FOREIGN KEY
prescription...	PRIMARY	PRIMARY KEY
room	chk_room_charge	CHECK
room	chk_room_status	CHECK
room	PRIMARY	PRIMARY KEY
room	uq_room_number	UNIQUE

Note: If this query is run after all constraints have been created, the result will include the constraints currently defined in the database.

8. Conclusion

The Hospital Management System database was designed and implemented using MySQL DDL commands. Eleven tables were created to manage the hospital's main activities.

Primary keys uniquely identify records, foreign keys establish relationships, and additional constraints help maintain data integrity. The tables are structured to reduce unnecessary duplication and support efficient retrieval of hospital information.

The database provides a foundation for inserting sample data, executing queries, generating reports, and managing hospital operations.